Answerspot — Shared API Client & Recommendation Island

Two files. The API client is the single dependency every wired page shares; the recommendation island is the first real consumer of its client-side half.

```
packages/shared-types/src/index.ts        # types shared API <-> web
apps/web/lib/api.ts                        # the shared client (server + browser)
apps/web/app/businesses/[id]/recommendation-list.tsx  # client island
```

`packages/shared-types/src/index.ts`

Single source of truth for the shapes crossing the API/web boundary. The API returns these; the web consumes them. Keep in sync with the Prisma models.

typescript

```ts
// ── Enums (mirror Prisma) ──
export type Plan = "STARTER" | "PRO" | "AGENCY";
export type Role = "OWNER" | "ADMIN" | "MEMBER";
export type Severity = "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
export type Sentiment = "POSITIVE" | "NEUTRAL" | "NEGATIVE" | "UNKNOWN";
export type PlatformKey = "CHATGPT" | "PERPLEXITY" | "GEMINI" | "AI_OVERVIEW";
export type RecommendationType =
  | "KEYWORD_GAP" | "SCHEMA_MARKUP" | "REVIEW_SIGNAL"
  | "CITATION_GAP" | "SENTIMENT_ISSUE" | "COMPETITOR_OVERTAKE";

// ── Domain ──
export interface Business {
  id: string;
  name: string;
  category: string;
  city: string;
  state: string;
  website?: string | null;
}

export interface ScoreBreakdown {
  appearanceRate: number;
  rankScore: number;
  sentimentScore: number;
  citationScore: number;
  weights: Record<string, number>;
}

export interface VisibilityScore {
  score: number;
  breakdown: ScoreBreakdown;
  computedAt: string;
}

export interface ScoreResponse {
  current: VisibilityScore | null;
  trend: { score: number; computedAt: string }[];
}

export interface RecommendationArtifact {
  kind: "code" | "text";
  content: string;
}
```

```typescript
export interface Recommendation {
  id: string;
  type: RecommendationType;
  severity: Severity;
  title: string;
  message: string;
  artifact: RecommendationArtifact | null;
  status: "OPEN" | "RESOLVED" | "DISMISSED";
}

export interface CompetitorRow {
  name: string;
  appears: number;
  total: number;
  you: boolean;
}

export interface Paginated<T> {
  data: T[];
  total: number;
  page: number;
  limit: number;
}
```

`apps/web/lib/api.ts`

Works in both contexts. On the server it reads the session cookie; in the browser it relies on the httpOnly cookie being sent automatically. One typed error class so every caller can branch on `.status`.

typescript

```ts
import type {
  Business, ScoreResponse, Recommendation, CompetitorRow, Paginated,
} from "@answerspot/shared-types";

const BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:4000/api/v1";

export class ApiError extends Error {
  constructor(message: string, public status: number, public body?: unknown) {
    super(message);
    this.name = "ApiError";
  }
}

// Resolve the auth header. On the server we read the cookie explicitly; in the
// browser the httpOnly cookie rides along automatically (credentials: include).
async function authHeaders(): Promise<Record<string, string>> {
  if (typeof window === "undefined") {
    // Server: dynamic import keeps next/headers out of the client bundle.
    const { cookies } = await import("next/headers");
    const token = cookies().get("session")?.value;
    return token ? { Authorization: `Bearer ${token}` } : {};
  }
  return {};
}

interface RequestOpts {
  method?: "GET" | "POST" | "PATCH" | "DELETE";
  body?: unknown;
  // Next fetch caching. Dashboards use "no-store"; static content can revalidate.
  cache?: RequestCache;
  next?: { revalidate?: number; tags?: string[] };
}

export async function apiFetch<T>(path: string, opts: RequestOpts = {}): Promise<T> {
  const headers: Record<string, string> = {
    "Content-Type": "application/json",
    ...(await authHeaders()),
  };

  const res = await fetch(`${BASE}${path}`, {
    method: opts.method ?? "GET",
    headers,
    body: opts.body ? JSON.stringify(opts.body) : undefined,
```

```
    credentials: "include",
    cache: opts.cache ?? "no-store",
    next: opts.next,
  });

  if (!res.ok) {
    let body: unknown;
    try { body = await res.json(); } catch { /* non-JSON error */ }
    const msg =
      res.status === 401 ? "unauthorized"
      : res.status === 403 ? "forbidden"
      : res.status === 404 ? "not_found"
      : `request_failed_${res.status}`;
    throw new ApiError(msg, res.status, body);
  }

  // 204 No Content
  if (res.status === 204) return undefined as T;
  return res.json() as Promise<T>;
}

// ── Typed endpoint wrappers (the surface pages actually call) ──
export const api = {
  business: {
    get: (id: string) => apiFetch<Business>(`/businesses/${id}`),
    list: (page = 1, limit = 20) =>
      apiFetch<Paginated<Business>>(`/businesses?page=${page}&limit=${limit}`),
    create: (input: Pick<Business, "name" | "category" | "city" | "state"> & { websit
      apiFetch<Business>(`/businesses`, { method: "POST", body: input }),
    score: (id: string) => apiFetch<ScoreResponse>(`/businesses/${id}/visibility-scor
    recommendations: (id: string) =>
      apiFetch<Recommendation[]>(`/businesses/${id}/recommendations`),
    competitors: (id: string) =>
      apiFetch<CompetitorRow[]>(`/businesses/${id}/competitors`),
  },
  scans: {
    trigger: (trackedQueryId: string, platformKeys?: string[]) =>
      apiFetch<{ queued: boolean }>(`/scans/trigger`, {
        method: "POST", body: { trackedQueryId, platformKeys },
      }),
  },
};
```

> Now the wired Home page's `apiGet` collapses to `api.business.get(id)` etc., and every future page shares one auth/error/caching implementation.

`apps/web/app/businesses/[id]/recommendation-list.tsx`

The client island the Home page imports. First real consumer of the client-side client. Handles its own interaction (expand/copy) and states (resolving, empty).

tsx

```tsx
"use client";

import { useState, useTransition } from "react";
import { AlertTriangle, Sparkles, ChevronRight, Copy, Check, CheckCircle2 } from "luc
import type { Recommendation, Severity } from "@answerspot/shared-types";
import { api, ApiError } from "@/lib/api";

const SEVERITY_STYLES: Record<Severity, string> = {
  CRITICAL: "bg-red-100 text-red-700 border-red-200",
  HIGH: "bg-orange-100 text-orange-700 border-orange-200",
  MEDIUM: "bg-amber-100 text-amber-700 border-amber-200",
  LOW: "bg-slate-100 text-slate-600 border-slate-200",
};

function RecCard({ rec, onResolved }: { rec: Recommendation; onResolved: (id: string)
  const [open, setOpen] = useState(false);
  const [copied, setCopied] = useState(false);
  const [pending, startTransition] = useTransition();
  const [error, setError] = useState<string | null>(null);

  const copy = async () => {
    if (!rec.artifact) return;
    try {
      await navigator.clipboard.writeText(rec.artifact.content);
      setCopied(true);
      setTimeout(() => setCopied(false), 1500);
    } catch {
      setError("Couldn't copy to clipboard.");
    }
  };

  const resolve = () => {
    startTransition(async () => {
      try {
        // PATCH /recommendations/:id { status: "RESOLVED" } (endpoint: M5).
        await api.business.recommendations; // placeholder ref; real call below
        await fetchResolve(rec.id);
        onResolved(rec.id);
      } catch (e) {
        setError(e instanceof ApiError ? "Couldn't update. Try again." : "Unexpected
      }
    });
  };
```

```
  return (
    <div className="border border-slate-200 rounded-xl p-4 bg-white hover:border-slat
      <div className="flex items-start gap-3">
        <div className="mt-0.5">
          {rec.severity === "HIGH" || rec.severity === "CRITICAL"
            ? <AlertTriangle size={18} className="text-orange-500" />
            : <Sparkles size={18} className="text-amber-500" />}
        </div>
        <div className="flex-1 min-w-0">
          <div className="flex items-center gap-2 flex-wrap">
            <h4 className="font-semibold text-slate-800 text-sm">{rec.title}</h4>
            <span className={`text-[10px] font-bold px-2 py-0.5 rounded-full border $
              {rec.severity}
            </span>
          </div>
          <p className="text-sm text-slate-500 mt-1 leading-relaxed">{rec.message}</p

          <div className="flex items-center gap-4 mt-2">
            {rec.artifact && (
              <button onClick={() => setOpen(!open)}
                className="text-xs font-semibold text-blue-600 hover:text-blue-700 in
                {open ? "Hide the fix" : "Show me the fix"}
                <ChevronRight size={14} className={`transition-transform ${open ? "ro
              </button>
            )}
            <button onClick={resolve} disabled={pending}
              className="text-xs font-semibold text-slate-400 hover:text-slate-600 di
              {pending ? "Marking…" : "Mark as done"}
            </button>
          </div>

          {error && <p className="text-xs text-red-600 mt-2">{error}</p>}

          {open && rec.artifact && (
            <div className="mt-2 relative">
              <pre className="text-[11px] bg-slate-900 text-slate-100 rounded-lg p-3
                {rec.artifact.content}
              </pre>
              <button onClick={copy}
                className="absolute top-2 right-2 text-slate-300 hover:text-white bg-
                {copied ? <Check size={14} /> : <Copy size={14} />}
              </button>
            </div>
```

```
          )}
        </div>
      </div>
    </div>
  );
}

// Thin call until the PATCH wrapper is added to lib/api.ts in M5.
async function fetchResolve(id: string): Promise<void> {
  const { apiFetch } = await import("@/lib/api");
  await apiFetch(`/recommendations/${id}`, { method: "PATCH", body: { status: "RESOLV
}

export function RecommendationList({ recommendations }: { recommendations: Recommenda
  const [items, setItems] = useState(recommendations);
  const onResolved = (id: string) => setItems((prev) => prev.filter((r) => r.id !== i

  if (items.length === 0) {
    return (
      <div className="bg-white rounded-2xl border border-slate-200 p-8 text-center">
        <CheckCircle2 size={28} className="text-green-500 mx-auto mb-2" />
        <p className="text-sm font-semibold text-slate-700">You&apos;re all caught up
        <p className="text-xs text-slate-400 mt-1">No open fixes right now. New ones
      </div>
    );
  }

  return (
    <div className="bg-white rounded-2xl border border-slate-200 p-6">
      <div className="flex items-center justify-between mb-4">
        <div className="flex items-center gap-2">
          <CheckCircle2 size={18} className="text-blue-600" />
          <h2 className="text-sm font-semibold text-slate-700 uppercase tracking-wide
            Top fixes to raise your score
          </h2>
        </div>
        <span className="text-xs text-slate-400">{items.length} open</span>
      </div>
      <div className="space-y-3">
        {items.map((r) => <RecCard key={r.id} rec={r} onResolved={onResolved} />)}
      </div>
    </div>
```

```
  );
}
```

How this ties together

- `packages/shared-types` is imported by both `apps/api` (validate its responses match) and `apps/web` (consume them). One definition, no drift.
- `apps/web/lib/api.ts` replaces the ad-hoc `apiGet` in the wired Home page. That page becomes: `const business = await api.business.get(params.id)` etc.
- `recommendation-list.tsx` is the client island the Home page renders. It now has real interactions (mark-as-done via PATCH), optimistic removal, and an empty state — none of which the mockup had.
- `@/lib/*` / `@answerspot/shared-types` path aliases go in `apps/web/tsconfig.json` and the root workspace config.

> Cleanup note: the placeholder `await api.business.recommendations;` line in `resolve()` is a no-op reference left to show where the M5 PATCH wrapper lands — remove it once `api.recommendations.resolve(id)` exists on the client.